```
!pip install -q transformers datasets tf-keras
```

```
import os

# --- CRITICAL FIX FOR TENSORFLOW 2.16+ ---
# This forces TensorFlow to use the legacy Keras 2 backend, which
# is compatible with Hugging Face Transformers.
os.environ["TF_USE_LEGACY_KERAS"] = "1"

import tensorflow as tf
import numpy as np
from datasets import load_dataset
from transformers import AutoTokenizer, TFAutoModelForSequenceClassification
from google.colab import files
```

```
BATCH_SIZE = 16
MAX_LEN = 256
LEARNING_RATE = 2e-5
EPOCHS = 2

# Verify that we are running with the fix
print(f"TensorFlow Version: {tf.__version__}")
try:
    # Check if legacy keras is active
    import tf_keras
    print("Success: Using Legacy Keras (tf-keras) compatibility.")
except ImportError:
    print("Warning: tf-keras not found. Code might fail.")
```

```
TensorFlow Version: 2.19.0
Success: Using Legacy Keras (tf-keras) compatibility.
```

```
print("Loading IMDB dataset...")
dataset = load_dataset("imdb")
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
```

```
Loading IMDB dataset...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
README.md:       7.81k/? [00:00<00:00, 727kB/s]
plain_text/train-00000-of-00001.parquet: 100%  21.0M/21.0M [00:00<00:00, 28.9MB/s]
plain_text/test-00000-of-00001.parquet: 100%  20.5M/20.5M [00:00<00:00, 74.8MB/s]
plain_text/unsupervised-00000-of-00001.p(…): 100%  42.0M/42.0M [00:00<00:00, 84.6MB/s]
Generating train split: 100%  25000/25000 [00:00<00:00, 36301.85 examples/s]
Generating test split: 100%  25000/25000 [00:00<00:00, 59919.75 examples/s]
Generating unsupervised split: 100%  50000/50000 [00:00<00:00, 75874.55 examples/s]
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 1.97kB/s]
config.json: 100%  483/483 [00:00<00:00, 16.3kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 1.93MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 3.63MB/s]
```

```
def tokenize_batch(batch):
    return tokenizer(
        batch["text"],
        padding="max_length",
        truncation=True,
        max_length=MAX_LEN
    )

print("Tokenizing data...")
tokenized_datasets = dataset.map(tokenize_batch, batched=True)
```

```
Tokenizing data...
Map: 100% 25000/25000 [00:41<00:00, 694.40 examples/s]
Map: 100% 25000/25000 [00:21<00:00, 1055.21 examples/s]
Map: 100% 50000/50000 [00:42<00:00, 1220.16 examples/s]
```

```
tokenized_datasets = tokenized_datasets.remove_columns(["text"])
tokenized_datasets = tokenized_datasets.rename_column("label", "labels")
tokenized_datasets.set_format(type="tensorflow", columns=["input_ids", "attention_mask", "labels"])
```

```
train_tf = tokenized_datasets["train"].to_tf_dataset(
    columns=["input_ids", "attention_mask"],
    label_cols=["labels"],
    shuffle=True,
    batch_size=BATCH_SIZE,
)

test_tf = tokenized_datasets["test"].to_tf_dataset(
    columns=["input_ids", "attention_mask"],
    label_cols=["labels"],
    shuffle=False,
    batch_size=BATCH_SIZE,
)
```

```
/usr/local/lib/python3.12/dist-packages/datasets/arrow_dataset.py:403: FutureWarning: The output of `to_tf_dataset` will cha
Old behaviour: columns=['a'], labels=['labels'] -> (tf.Tensor, tf.Tensor)
             : columns='a', labels='labels' -> (tf.Tensor, tf.Tensor)
New behaviour: columns=['a'],labels=['labels'] -> ({'a': tf.Tensor}, {'labels': tf.Tensor})
             : columns='a', labels='labels' -> (tf.Tensor, tf.Tensor)
  warnings.warn(
```

```
print("Initializing Model...")
model = TFAutoModelForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2,
    from_pt=True
)
```

```
Initializing Model...
pytorch_model.bin: 100% 268M/268M [00:01<00:00, 179MB/s]
TensorFlow and JAX classes are deprecated and will be removed in Transformers v5. We recommend migrating to PyTorch classes
Some weights of the PyTorch model were not used when initializing the TF 2.0 model TFDistilBertForSequenceClassification: ['
- This IS expected if you are initializing TFDistilBertForSequenceClassification from a PyTorch model trained on another tas
- This IS NOT expected if you are initializing TFDistilBertForSequenceClassification from a PyTorch model that you expect to
Some weights or buffers of the TF 2.0 model TFDistilBertForSequenceClassification were not initialized from the PyTorch mode
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
```

```
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.losses import SparseCategoricalCrossentropy
from tensorflow.keras.metrics import SparseCategoricalAccuracy

model.compile(
    optimizer=Adam(learning_rate=LEARNING_RATE),
    loss=SparseCategoricalCrossentropy(from_logits=True),
    metrics=[SparseCategoricalAccuracy(name="accuracy")]
)
model.summary()
```

```
Model: "tf_distil_bert_for_sequence_classification"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 distilbert (TFDistilBertMa  multiple                  66362880
 inLayer)

 pre_classifier (Dense)      multiple                  590592

 classifier (Dense)          multiple                  1538

 dropout_19 (Dropout)        multiple                  0

=================================================================
Total params: 66955010 (255.41 MB)
Trainable params: 66955010 (255.41 MB)
Non-trainable params: 0 (0.00 Byte)
_________________________________________________________________
```

```
print("Starting Training...")
history = model.fit(
    train_tf,
    validation_data=test_tf,
    epochs=EPOCHS
)
```

```
Starting Training...
Epoch 1/2
1563/1563 [==============================] - 900s 557ms/step - loss: 0.2852 - accuracy: 0.8820 - val_loss: 0.2187 - val_accu
Epoch 2/2
1563/1563 [==============================] - 868s 556ms/step - loss: 0.1666 - accuracy: 0.9380 - val_loss: 0.2188 - val_accu
```

```
print("Evaluating on Test Set...")
eval_res = model.evaluate(test_tf)
print(f"Final Test Loss: {eval_res[0]}")
print(f"Final Test Accuracy: {eval_res[1]}")
```

```
Evaluating on Test Set...
1563/1563 [==============================] - 222s 142ms/step - loss: 0.2188 - accuracy: 0.9120
Final Test Loss: 0.21881583333015442
Final Test Accuracy: 0.9120000004768372
```

```
# Define the folder name and the zip file name
save_directory = "distilbert_imdb_tf_model"
zip_filename = f"{save_directory}.zip"

# Save the model to the specified directory
print(f"Saving the model to: {save_directory}")
model.save_pretrained(save_directory)

# Zip the entire model folder
# This is crucial because Colab cannot download folders directly,
# and zipping large files improves reliability.
print(f"Zipping the model folder: {save_directory}")
!zip -r {zip_filename} {save_directory}

# 2. Trigger the local download
print(f"Starting download of {zip_filename}...")
files.download(zip_filename)

# Optional: You can also verify the file size before downloading
print(f"\nSize of the zip file:")
!du -h {zip_filename}
```

```
Saving the model to: distilbert_imdb_tf_model
Zipping the model folder: distilbert_imdb_tf_model
  adding: distilbert_imdb_tf_model/ (stored 0%)
  adding: distilbert_imdb_tf_model/tf_model.h5 (deflated 8%)
  adding: distilbert_imdb_tf_model/config.json (deflated 42%)
Starting download of distilbert_imdb_tf_model.zip...

Size of the zip file:
236M    distilbert_imdb_tf_model.zip
```